

Simultaneous Reverse Engineering & Feature Identification with Machine Translation ...

By Hugo Espiritu, Hailey Gage, Nick Ramos, Suleyman Olcay Polat

Outline

- Problem Description
- The Approach
- Creating the Data Set
- Dataset Imbalance
- Machine Learning Classifiers Results

Problem Description

- Modify off-the-shelf machine learning models to decompile binary code with an emphasis on feature identification.
- We adapt ideas from recurrent neural net (RNN) decompilation (binary to C) for the Seq2Seq neural machine translation model (NMT).
 - Seq2Seq will have an emphasis on feature identification and test the effectiveness on a curated parallel corpus of source and binary code.

Our Approach

- Most reverse engineering tools are inefficient, incomplete, and limited in scope.
 - In particular, disassembly listings have a very low abstraction level and require an analyst to manually map assembly instructions to higher level abstractions.
- The performance of machine translation models are giving hope for using ML for decompiling.
- We have chosen the following set of ML classifiers:
 - Random Forest
 - Support Vector Machine
 - Recurrent Neural Net
 - Seq2Seq encoder-decoder framework

Creating Data Set

- There are 6 different optimization levels for c compiling
- We only use O0 optimization for our dataset creation

option	optimization level	execution time	code size	memory usage	compile time
-O0	optimization for compilation time (default)	+	+	-	-
-O1 or -O	optimization for code size and execution time	-	-	+	+
-O2	optimization more for code size and execution time	--		+	++
-O3	optimization more for code size and execution time	---		+	+++
-Os	optimization for code size		--		++
-Ofast	O3 with fast none accurate math calculations	---		+	+++

+increase ++increase more +++increase even more -reduce --reduce more ---reduce even more

Data Pre-Processing

Step1) Disassembled data using BAP

Step2) Found function starts using Byteweight tool

Step3) Label function start bytes as 1, all other bytes as 0

Dataset Imbalance

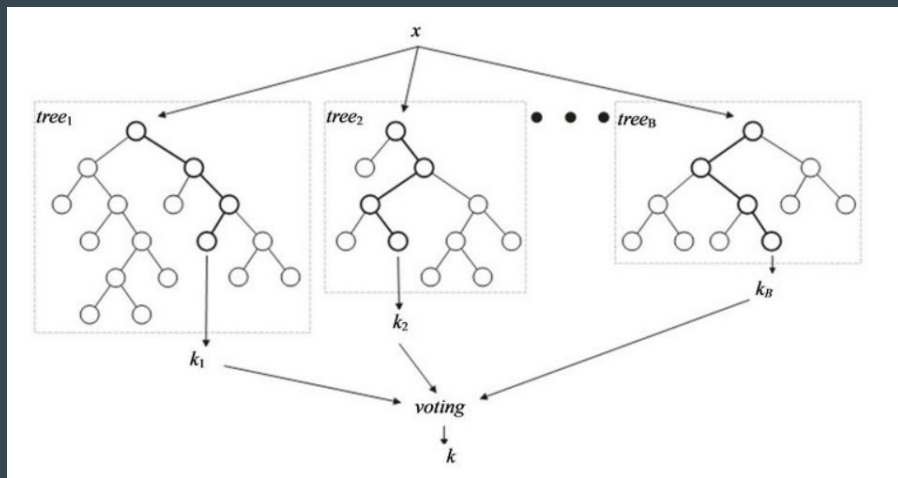
Dataset imbalance occurs when one or more of the classes in your dataset occurs at a lower ratio when compared to your other classes. This causes models to ignore under represented class and still get a very high accuracy. In our case class ratio was roughly 1 to 1000.



Loss Function For Imbalanced Datasets

Loss function calculates the distance between expected output of a model and the actual output. This distance is used to calculate weight updates for the model. Using the reverse of class ratios as a weight for the loss function helps the models work on imbalanced datasets.

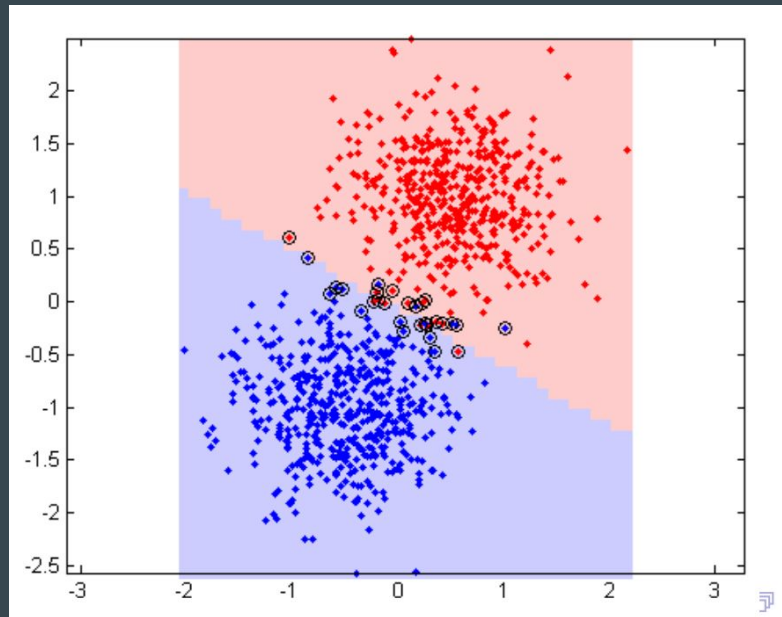
Random Forest



- Not modified to handle class imbalance
- 3514047 True Negative
- 0 False Positive
- 9487 False Negative
- 0 True Positive

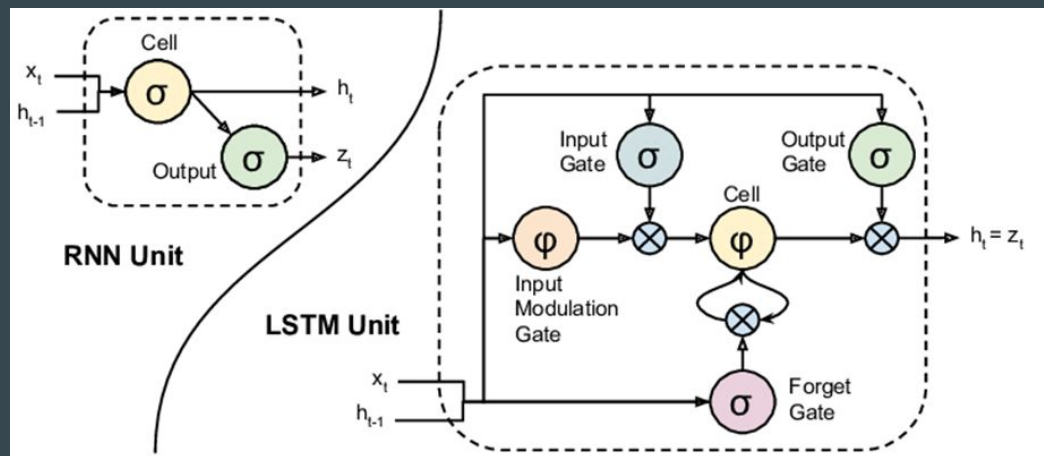
Support Vector Machine

- 2590980 True Negative
- 923067 False Positive
- 7244 False Negative
- 2193 True Positive
- 23% of the function starting points found
- 923067 bytes miss classified as function starts



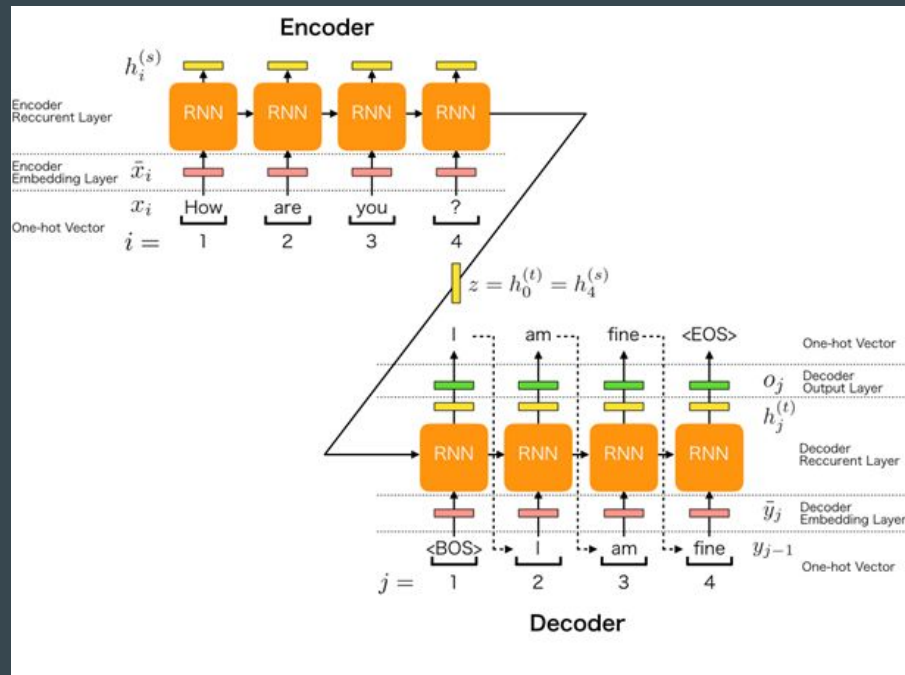
Recurrent Neural Net

- Not modified to handle class imbalance
- 3514047 True Negative
- 0 False Positive
- 9487 False Negative
- 0 True Positive



Seq2Seq

- 2127520 True Negative
- 1386527 False Positive
- 6242 False Negative
- 3245 True Positive
- 34% of the function starting points found
- 1386527 bytes miss classified as function starts



Conclusion & Future Work

Our work was inconclusive on the best model for working on binary data.

- Improve the dataset with more data and more classes to counter class imbalance
- Try different boosting algorithms for training (adaboost, ensemble ...)

Questions?